GESTURE CONTROL / VIDEO PLAYER

A course project report submitted in partial fulfilment of the requirement of

SMART SYSTEM DESIGN

by

Shaik Naved Ahmed (2303A51302)

Shaik Nishaaj (2303A51613)

Vutukuru Sai Anand (2303A51310)

Shaik Afroz (2303A51983)

Under the guidance of

Mr. B.Girirajan

Asst. Professor, Department of ECE

Mr. Sacheth Mahipal

SR UNIVERSITY

ABSTRACT

This report presents a thorough examination of the state-of-the-art in hand gesture recognition technology and its application in enhancing human-computer interaction (HCI) with laptops. As traditional input methods may become limiting in certain scenarios, gesture-based interaction offers a promising alternative by allowing users to control laptops through natural hand movements. The report begins by providing an overview of the evolution of HCI, emphasizing the importance of intuitive and efficient interaction paradigms. The core focus of the report is on the various techniques and technologies employed in hand gesture recognition systems, ranging from traditional computer vision methods to more recent advancements using deep learning approaches. It explores the strengths and limitations of different gesture recognition algorithms, considering factors such as accuracy, real-time performance, and adaptability to diverse user environment.Furthermore, the report investigates the practical applications of hand gestures in controlling laptop functionalities, including navigation, manipulation of virtual objects, and execution of commands. It delves into case studies and experimental setups that highlight successful implementations of gesture-based interaction in diverse domains such as gaming, presentations, and creative design.In addition, the challenges and potential issues associated with gesture-based HCI for laptops are discussed, addressing concerns related to user fatigue, environmental factors, and the need for standardized gesture libraries. The report also considers the user experience (UX) aspect examining how the design of gesture-based interfaces can impact user satisfaction and productivity.Lastly, the report provides insights into future trends and developments in the field, including emerging technologies and novel applications that could further revolutionize the landscape of gesture-based interaction with laptops. The synthesis of this comprehensive analysis aims to inform researchers, developers, and practitioners about the current state of hand gesture recognition technology and its implications for enhancing laptop usability through intuitive and natural interaction.

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

For decades, keyboards and mouse have been the main input devices for computers. However, with the rising popularity of ubiquitous and ambient devices, (i.e. PlayStations), and equipment which allows users to grasp virtual objects, hand or body gesture are becoming essential: gesture controls have become central to the human computer interaction system.

Gesture control technology is based on gesture recognition. Gesture recognition can be seen as a way for computers to begin to understand human body language. Compared to the primitive user interfaces, such as keyboard and mouse, it builds a richer bridge between the computers and humans. Gesture control devices recognize and interpret human body movements, allowing the user to interact with a computer system.

Leap Motion is a hardware sensor that detects finger movement and allows users to interact with a computer by waving their fingers and hands with no need for touch or the use of a mouse. Our motivation is to implement such a device which makes interaction more convenient.

1.2 OBJECTIVES

The objective of this project is to implement a low cost, reliable and scalable gesture control laptop that can be used to perform any regular tasks by using some simple hand gestures

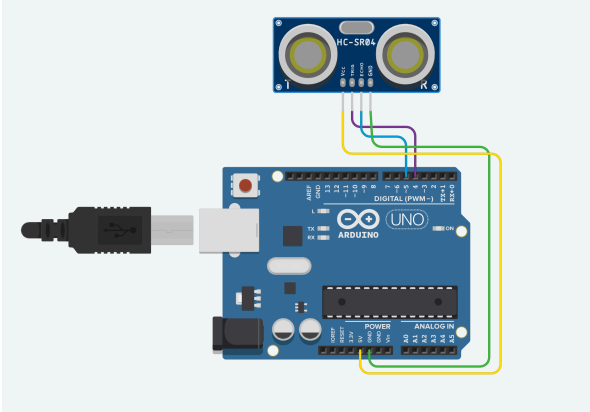

Fig 1.2.1 CIRCUIT DIAGRAM

Fig 1.2.2 PROJECT PICTURE (WITH PDS)

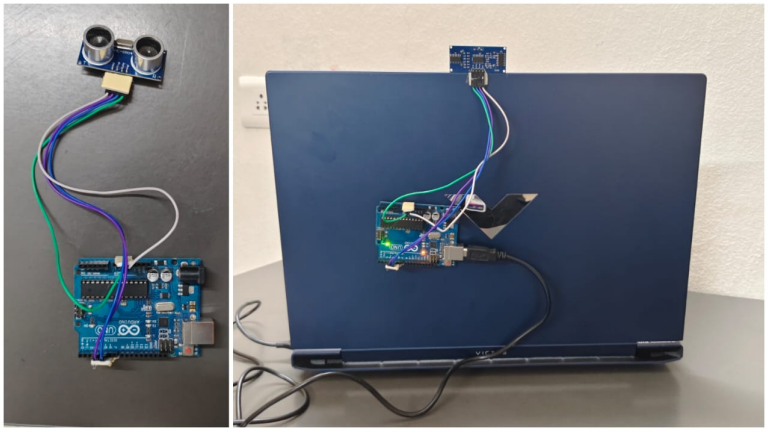

Fig 1.2.3 PROJECT PICTURE (WITHOUT PDS)

CHAPTER 2
PROJECT DESCRIPTION

We will place two Ultrasonic (US) sensors on top of our monitor and will read the distance between the monitor and our hand using Arduino , based on this value of distance we will perform certain actions. To perform actions on our computer we use Python **pyautogui** library. The commands from Arduino are sent to the computer through serial port (USB). This data will be then read by python which is running on the computer and based on the read data an action will be performed.

2.1 HARDWARE DESCRIPTION

ARDUINO : Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, - and turn it into an output - activating a motor, turning on an LED. Here we use Arduino to read the distance of hand from the US sensor.

Fig 2.1.1 Arduino UNO

ULTRASONIC SENSORS : Ultrasonic sensors are great tools to measure distance and detect objects without any actual contact with the physical world. In this project, we will use the **HC-SR04 Ultrasonic Sensor with Arduino** to determine the distance of hand from the sensor.

Fig 2.1.2 Ultrasonic Sensor

2.2 CONNECTIONS:

The connections are to be given as follows and the details are discussed more clearly in the design analysis part. To control the PC with Hand Gestures, just connect the two Ultrasonic sensors with Arduino. The two US sensors are connected to Digital pins 2,3,4 and 5 and are powered by +5V pin. The trigger pins are output pin and Echo pins are input pins.The Arduino can be connected to the PC/Laptop for powering the module and also for Serial communication. Once the connections are done place them on your monitor as shown below

CHAPTER 3

CIRCUIT DIAGRAM AND DESCRIPTION

3.1 WORKING

Gesture controlling is based on specifying hand position from the ultrasonic sensor. For processing the raw data, a micro-controller is essential; for that we use Arduino UNO board. Via USB connection the micro-controller transfers the processed and calculated distance value which is provided by the sensor. The data which is send by the sensor is processed in the software in PC where all the calculations are performed and the data is matched with the predefined conditions (gesture resolution). The distance between the sounds is send and detection of reflect back sound wave is calculated by the microcontroller. To run this model, the python code should run on python GUI first. It matches gestures with predefined conditions and prints on python output shell. By reading the value of distance we can arrive at certain actions to be controlled with gestures.

- **Action 1:** When right hand is placed up before the sensor at a particular far distance then the video should Fast Forward one step.
- **Action 2:** When left hand is placed up before the sensor at a particular distance then the video should Rewind one step.

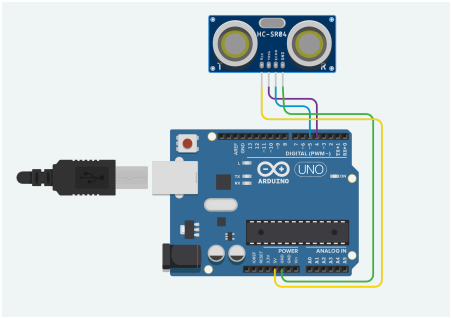

Fig 3.1.1 Circuit diagram

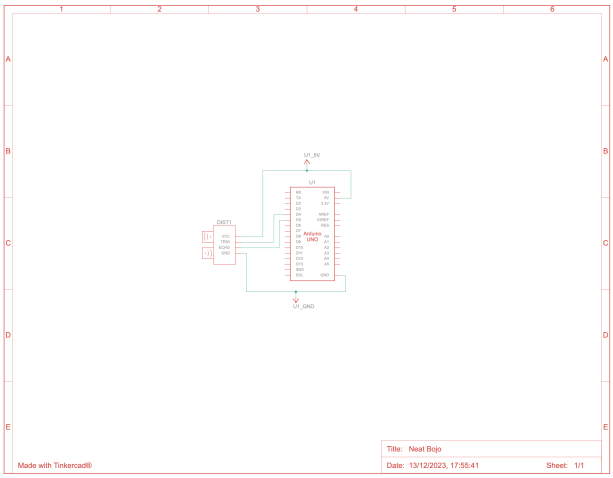

Fig 3.1.2 Schematic diagram

3.2 RESULTS

HAND	GESTURE	VIDEO ACTION
Right	Moved away from sensor	Rewind
Right	Moved towards sensor	Fast forward

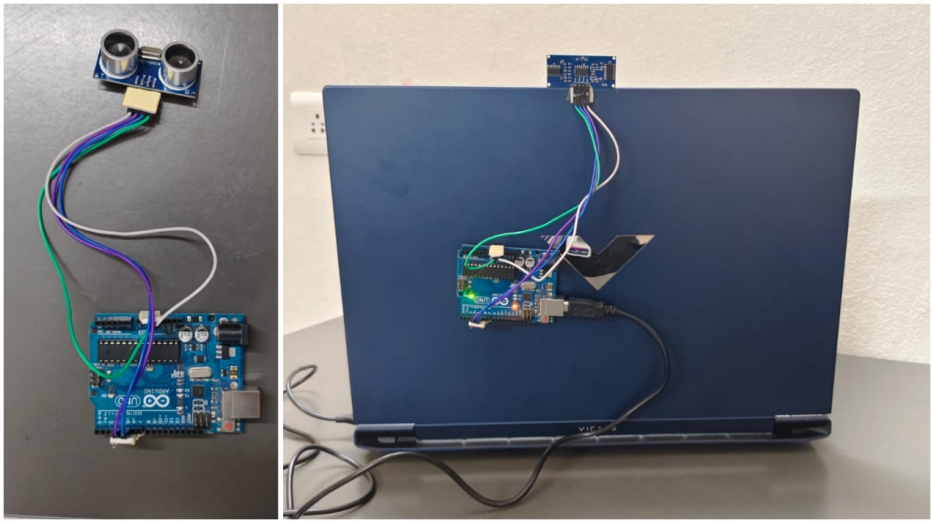

Fig 3.2.1 Experimental result

3.3 ADVANTAGES

The desktop computing paradigm limits the users' flexibility by forcing them to interact using a 2-Degree-Of-Freedom device (the mouse), while they are used to interacting with the physical world in much more differentiated way. Gestures allow the user to handle multiple points of input and even define several parameters at once. They are, therefore, a more natural form of communication.

Also, they do not require any additional devices: the command and even its parameters can be specified by a simple hand movement. Input devices narrow down the user's possibilities of interaction, for example a pen or a mouse limiting the potential forms of input to single-touch interaction. Gestures that are performed with the user's hands however, can be versatile and do not have these constraints. Gesture-controlled presentations were not only perceived as more enjoyable by the presenter but also as more attractive by the audience. The presenters were able to make eye contact more often and to use their body language to convey information.

3.4 DISADVANTAGES

This kind of input also raises issues that are not relevant with traditional input. On the user's side, these problems are to learn, to remember and to accurately execute gestures. The developer has to provide a system that correctly recognizes these gestures. The developer not only has to ensure that gestures are quickly and correctly recognized, but also has to provide a guide that allows a rapid and easy learning of these gestures.The teaching of multi-touch and mid-air gestures is more difficult than that of single-touch gestures. In the case of the latter, the hand posture is irrelevant - users only need to follow a path correctly to perform a comman

CHAPTER 4

CONCLUSION

Immersive gaming technology: Gestures may be used to control interactions with the gaming console and give a more interactive and immersive experience. Control through facial gestures: This technology can be used for applications with even more precision like recognizing face gestures Remote control: By using gesture recognition, it is possible to use hand alone as a remote control for various devices. Home Appliances control: It is possible to extend the gesture recognition technology to control the household appliances.

Future Scope

- It is one of the easiest ways of interaction between human and computer.
- It is a cost effective model which is only based on Arduino UNO and ultrasonic sensor.
- The python IDE allows a seamless integration with Arduino UNO in order to achieve different processing and controlling method for creating new gesture control solution.
- Additional gesture recognition opportunities exist in medical applications where, for health and safety reasons, a nurse or doctor may not be able to touch a display but still needs to control a system.

APPENDIX

Arduino Code:

```
const int trigger1 = 2; //Trigger pin of 1st Sensor

const int echo1 = 3; //Echo pin of 1st Sesnor

const int trigger2 = 4; //Trigger pin of 2nd Sesnor

const int echo2 = 5;//Echo pin of 2nd Sesnor

long time_taken;

int dist,distL,distR;

void setup( ) {

Serial.begin(9600);

pinMode(trigger1, OUTPUT);

pinMode(echo1, INPUT);

pinMode(trigger2, OUTPUT);

pinMode(echo2, INPUT);

}

/*###Function to calculate distance###*/

void calculate_distance(int trigger, int echo)

{

digitalWrite(trigger, LOW);

delayMicroseconds(2);

digitalWrite(trigger, HIGH);
```

```
delayMicroseconds(10);

digitalWrite(trigger, LOW);

time_taken = pulseIn(echo, HIGH);

dist= time_taken*0.034/2;

if (dist>50)

dist = 50;

}

void loop() { //infinite loopy

calculate_distance(trigger1,echo1);

distL =dist; //get distance of left sensor

calculate_distance(trigger2,echo2);

distR =dist; //get distance of right sensor

//Uncomment for debudding

/*Serial.print("L=");

Serial.println(distL);

Serial.print("R=");

Serial.println(distR);

*/

//Pause Modes -Hold

if ((distL >40 && distR>40) && (distL <50 && distR<50)) //Detect both hands

{Serial.println("Play/Pause"); delay (500);}
```

```
calculate_distance(trigger1,echo1);
distL =dist;
calculate_distance(trigger2,echo2);
distR =dist;
//Control Modes
//Lock Left - Control Mode
if (distL>=13 && distL<=17)
{
  delay(100); //Hand Hold Time
  calculate_distance(trigger1,echo1);
  distL =dist;
  if (distL>=13 && distL<=17)
  {
    Serial.println("Left Locked");
    while(distL<=40)
    {
      calculate_distance(trigger1,echo1);
      distL =dist;
      if (distL<10) //Hand pushed in
      {Serial.println ("Vup"); delay (300);}
      if (distL>20) //Hand pulled out
```

```
      {Serial.println ("Vdown"); delay (300);}
    }
  }
}
//Lock Right - Control Mode
if (distR>=13 && distR<=17)
{
  delay(100); //Hand Hold Time
  calculate_distance(trigger2,echo2);
  distR =dist;
  if (distR>=13 && distR<=17)
  {
    Serial.println("Right Locked");
    while(distR<=40)
    {
      calculate_distance(trigger2,echo2);
      distR =dist;
      if (distR<10) //Right hand pushed in
      {Serial.println ("Rewind"); delay (300);}
      if (distR>20) //Right hand pulled out
      {Serial.println ("Forward"); delay (300);}
```

```
  }
}
}
delay(200);
}
```

Python Code:

```
import serial #Serial imported for Serial communication
import time #Required to use delay functions
import pyautogui
ArduinoSerial = serial.Serial('com18',9600) #Create Serial port object called arduinoSerialData
time.sleep(2) #wait for 2 seconds for the communication to get established

while 1:
    incoming = str (ArduinoSerial.readline()) #read the serial data and print it as line
    print incoming

    if 'Play/Pause' in incoming:
        pyautogui.typewrite(['space'], 0.2)

    if 'Rewind' in incoming:
```

```
    pyautogui.hotkey('ctrl', 'left')

if 'Forward' in incoming:
    pyautogui.hotkey('ctrl', 'right')

 if 'Vup' in incoming:
    pyautogui.hotkey('ctrl', 'down')

 if 'Vdown' in incoming:
    pyautogui.hotkey('ctrl', 'up')

 incoming = "";z
```